

flashjet

GPU-native jet clustering with the full merge history

Chirayu Gupta

ML4Jets 2026 · Vienna · September 2026

Benchmarks in this talk use **ATLAS Open Data** (CC0) and analytic/toy closures — freely presentable,
no approval required.

The problem: clustering is the CPU step in a GPU pipeline

Jet clustering sits on the critical path of nearly every jet-level ML pipeline — and is almost always done **on the CPU, off the accelerator**.

Four-momenta are copied to the host, clustered, copied back.

That round trip dominates when:

- ▶ training loops revisit the same events for **many epochs**;
- ▶ jets are **reclustered** under many systematic or hyper-parameter variations;
- ▶ the clustering itself must sit **inside** a differentiable pipeline.

flashjet implements the generalised- k_t family (anti- k_t , k_t , Cambridge–Aachen) **natively on the GPU**.

Data never leaves device memory during clustering.

Written in Triton/PyTorch. One entry point, runs unchanged on CPU and GPU.

What it does

Clusters a batch of jets or events at once, directly in GPU memory.

Returns **two** things:

1. the particle $\rightarrow$ jet assignment;
2. the **complete merge history** — the full binary tree of which pair merged, and at what distance.

Why the history matters. Substructure observables — groomed mass, z_g , R_g , Lund coordinates, exclusive subjects — become **cheap reads of that tree**.

No second clustering pass.

Returned	Meaning
jet_idx, n_jets	particle-to-jet assignment
hist_p1, hist_p2, hist_child	which pair merged into what
hist_d	the d_{ij} at each merge step

The interface

One function. The algorithm is an argument; the backend is chosen automatically.

```
out = flashjet.cluster(p4, mask, R=1.0, algorithm="antikt")
#   p4      : (B, N, 4) torch tensor  (px, py, pz, E)
#   mask    : (B, N) bool              (ragged -> padded batch)
#   algorithm : "antikt" | "kt" | "cambridge"  (or p=... for generalised-kt)

out.n_jets          # (B,)          jets found per entry
out.jets_p4(p4)      # (B, J, 4) jet four-momenta  (differentiable)
out.splitting_scales()# sqrt(d12), sqrt(d23), ...
out.exclusive_jets(n_jets=3)          # F1
out.groomed_jets(p4, R, z_cut=0.1, beta=0.0)  # F2  soft drop / mMDT
out.lund_coordinates(p4, R)            # F3  primary Lund plane
```

Same call on CPU and GPU. jets_p4 is a scatter_add of constituents, so gradients flow through the clustering.

What is implemented

Clustering (kernels)

- ▶ generalised- k_t : anti- k_t ($p = -1$), k_t ($p = +1$), C/A ($p = 0$)
- ▶ batched, padded/ragged input via a mask
- ▶ three backends, auto-dispatched:
fused register kernel ($N \leq 16$) $\rightarrow$ scratch
NN-array Triton kernel ($N \leq 16384$) $\rightarrow$ torch
- ▶ full merge history recorded in-kernel
- ▶ jet-regime (batch of jets) *and* event-regime (whole events)

Substructure — reads of the history

- ▶ **F1** exclusive k_t subjets
`exclusive_jets_from_history` — undo the last merges to expose n subjets, or cut at d_{cut}
- ▶ **F2** soft drop / mMDT / mass-drop
`groom_from_history` — walk the harder branch until $z > z_{\text{cut}}(\Delta R/R)^\beta$
- ▶ **F3** primary Lund coordinates
`lund_coordinates_from_history` — $(z, \Delta R, k_t, \ln 1/\Delta R, \ln k_t, d)$ per split

All three are **pure-torch post-reads**: no kernel changes, CPU/GPU identical, negligible cost.

Status: a working prototype

flashjet is a **prototype**. The physics is validated and the speedups are real, but it is **not** production software: it is a Python/Triton library, not integrated into any experiment's reconstruction framework.

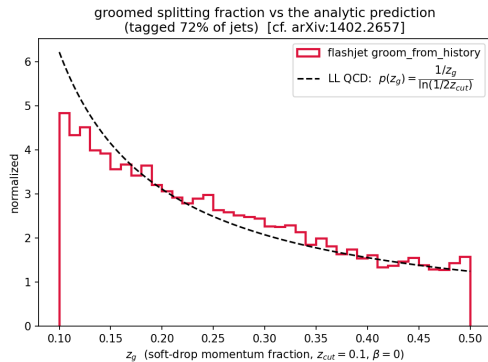
What is solid

- ✓ 129 unit tests, incl. GPU-only paths
- ✓ agrees with FastJet jet-by-jet
- ✓ reproduces analytic predictions
- ✓ reproduces a published experiment's own reconstructed jets
- ✓ all three algorithms, R-scan, multiplicity scan

What it is not (yet)

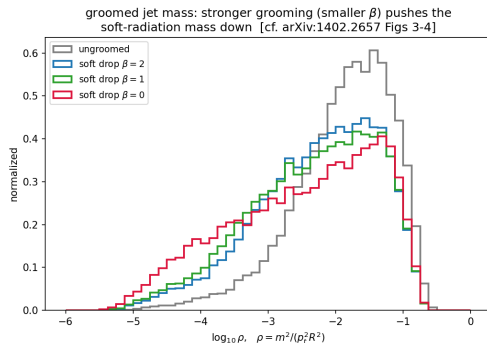
- ✗ not inside experiment software (needs a C++/CUDA rewrite)
- ✗ CPU baseline is the **Python** FastJet binding, not C++ FastJet
- ~ GPU is **largely idle** — launch configuration tuned on older cards
- ✗ no pile-up study; grooming not yet timed

The history is right: closures against analytic predictions



Soft-drop z_g follows the $1/z$ DGLAP form — predicted at leading-log, recovered from the C/A history.

Toy-shower input, so the *prediction* is unambiguous. These test the substructure decoders, not the event generator.

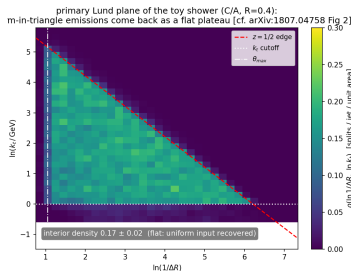


Groomed-mass ordering in β : larger β grooms less, so the mass peak moves up — reproduces Soft Drop Figs. 3–4.

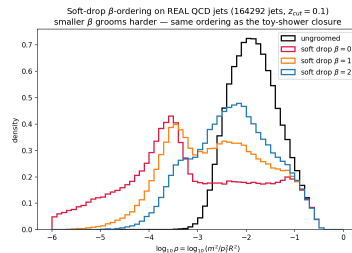
Clustering geometry and the Lund plane



anti- k_t jet areas $\rightarrow \pi R^2$: the known "rigid cone" property.



Lund plane populated uniformly inside the kinematic triangle.



β -family ordering on a toy shower.

Independent **NumPy tree-walks** pin every decoder; the d -channel of the Lund output ties *exactly* to `splitting_scales`.

The benchmark: what, on what, how

Data — ATLAS Open Data (CC0, no approval needed)

ATLAS Top Tagging Open Data

(ATL-PHYS-PUB-2022-039): 2.48 M large- R jets, **calorimeter-cluster constituents** stored per jet.

sample	jets	$\langle n_{\text{const}} \rangle$
boosted top	30 000	59.2
QCD (light q/g)	30 000	52.4

Multiplicity here is *higher* than typical small- R jet samples — these are boosted fat jets.

Compared against

- ▶ **FastJet vectorised** (awkward interface) — the *fair* CPU baseline
- ▶ **FastJet per-jet** (classic loop) — shown for reference

Discipline

1. extract constituents once, save;
2. cluster on GPU, time it, save the *same* events;
3. cluster those saved events with FastJet;
4. **check they agree** — else the timing is meaningless.

Correctness first: 150/150 points agree

The full grid: **3 algorithms** $\times$ **5 radii** $\times$ **5 multiplicity bins** $\times$ **2 samples**, 20 000 jets per bin.

Check	Result
number of jets found, vs FastJet	100.000 % at every one of 150 points
leading-jet p_T within 10^{-4} rel.	1.0000 at 148/150 points
median relative difference	$\sim 7 \times 10^{-8}$ (float32 precision)

k_t and C/A had never before been checked against FastJet on real detector data — only against NumPy tree-walks in unit tests. They now agree across the whole radius $\times$ multiplicity grid.

The two exceptions, stated honestly: both are the *same single jet* (97 constituents, 6 jets found by both). One ~ 29 GeV cluster is assigned to a different jet — a **boundary assignment at the R edge**, not a merge-ordering error. Jet counts still match; the leading jet is never ambiguous (790 GeV gap).

It also reproduces ATLAS's own reconstructed jets

A stronger test than agreeing with FastJet: the *2020 ATLAS Jet Reconstruction* open dataset ships both **calorimeter clusters** and ATLAS's own RecoJets_R4.

Cluster the ~ 600 clusters per event ourselves, compare to what ATLAS reconstructed:

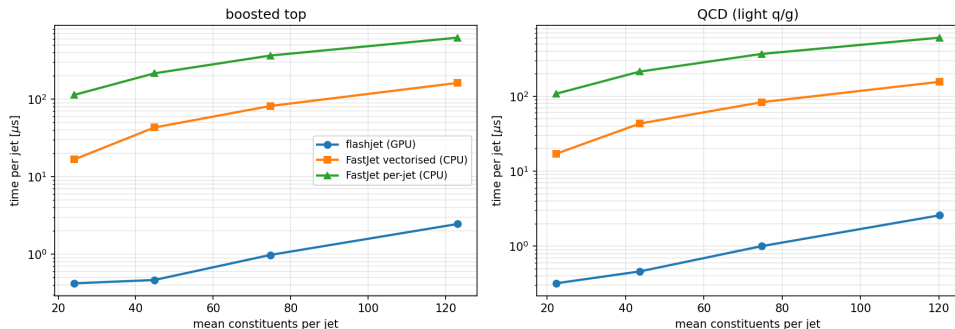
Comparison	Result
n_{jets} vs RecoJets_R4	100.0 % match
mean jets/event	10.96 vs 10.96
leading p_T vs FastJet	within 10^{-4}

This closes the algorithm against **the experiment's own published output**, not against another implementation of the same algorithm.

599.3 clusters/event — the event regime, where the problem is $O(N^2)$.

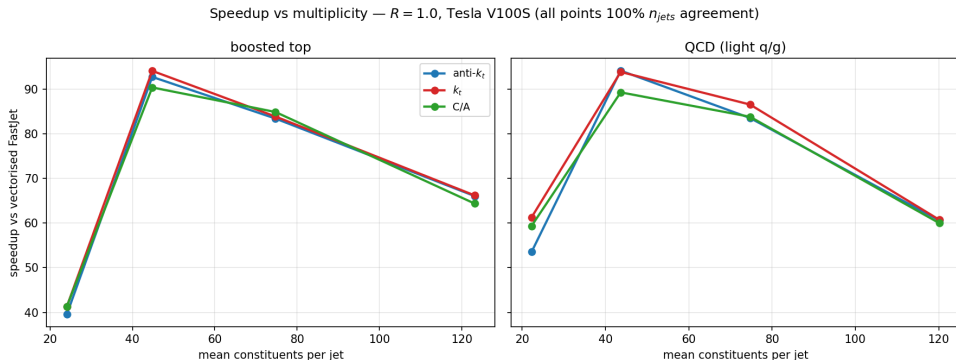
Time per jet

ATLAS Top Tagging Open Data — anti- k_t , $R = 1.0$, Tesla V100S



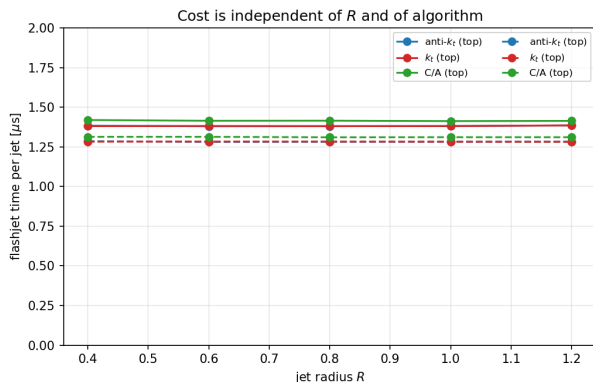
Log scale — each gridline is $10\times$. flashjet stays close to flat while both FastJet interfaces climb steeply with multiplicity. The gap is **two orders of magnitude** against the per-jet interface, and about $40\text{--}100\times$ against the vectorised one.

Speedup grows with jet complexity



All three algorithms, both samples, $R = 1.0$. Every point has **100 % n_{jets} agreement**. The busier the jet, the bigger the win — which is the regime that matters for boosted-object physics.

Cost is independent of R — and of the algorithm



Across $R = 0.4 \rightarrow 1.2$:

1.3797–1.3845 $\mu\text{s}/\text{jet}$. A **0.3 % spread** over a $3\times$ range in R .

R changes *which* pairs merge, not *how many* distances are computed.

Across algorithms: anti- k_t / k_t / C/A agree to within $\sim 4\%$ at every multiplicity — they share the kernel and differ only in the exponent.

k_t and C/A timing is **free**.

The numbers

sample	alg	$\langle n \rangle$	flashjet [$\mu\text{s}/\text{jet}$]	FJ vec. [$\mu\text{s}/\text{jet}$]	FJ per-jet [$\mu\text{s}/\text{jet}$]	speedup
top	anti- k_t	24.1	0.427	16.5	119.9	39×
top	anti- k_t	44.8	0.464	43.0	214.9	93 ×
top	C/A	74.7	1.000	84.8	361.9	85×
top	C/A	123.1	2.565	165.1	621.9	64×
qcd	anti- k_t	22.2	0.346	17.5	110.1	51×
qcd	anti- k_t	43.6	0.463	45.7	220.0	99 ×
qcd	anti- k_t	74.7	1.215	89.8	376.3	74×
qcd	anti- k_t	120.2	2.673	154.9	618.2	58×

39–99× over vectorised FastJet (median 64×), 100 % jet-count agreement everywhere — on a **Tesla V100S**.

$R = 1.0$ rows shown; the full 150-point grid is in the backup.

Caveats — read the speedups with these

1. **Hardware.** These are **V100S** numbers. A current-generation card (H100) is roughly a further $2\times$ on identical input, so treat these as a *lower bound* — not as this library's ceiling.
2. **The CPU baseline is Python.** Every number compares against FastJet's *Python* binding. The production tool is C++. Until that is measured, the honest caveat is “*some of this could be Python overhead*”. **This is the single most valuable missing test.**
3. **Large- R jets.** These are boosted fat jets ($R=1.0$ -scale, ~ 50 – 120 constituents), not small- R jets.
4. **Batch regime.** The advantage comes from clustering *many* jets at once. A per-event framework may not be able to exploit that — it needs measuring, not assuming.

[PLACEHOLDER] ParT + C/A features — current status

Slide to be finalised. Numbers below are the current measured state as of 4 Sep 2026; the follow-up run is still in progress.

The idea. If flashjet hands back the merge tree for free, feed it to the tagger: add per-particle Cambridge/Aachen branch-point features to Particle Transformer on JetClass.

The result: no gain. Matched 480k checkpoint, 20.05 M-jet test set:

metric	baseline	+ C/A (5 per-particle)	Δ
test accuracy	85.625 %	85.524 %	-0.102
mean 1-vs-QCD AUC	0.98981	0.98970	-0.00011

All nine classes marginally worse. Magnitudes sit inside the ~ 1.3 -point seed spread, so the honest statement is “**indistinguishable from baseline, with a consistent negative sign**” — not “C/A hurts”. An earlier +1.3 at 20k iterations was a **training transient**, not a durable effect.

[PLACEHOLDER] Why it failed — and the fix

Diagnosed, not just observed.

The feature values are provably correct (0 soft-side violations in 274 192 entries). The problem is structural:

Measured group size is exactly 1.00 — the features carry *zero* prong-grouping information.

In a C/A tree a particle almost always merges into something larger, so “am I the softer child?” is true at its *own first merge* — the walk never climbs, and the node it lands on is unique to that particle.

Consequence. Prong membership is a property of a *pair*, not of a particle. The pairwise redesign was built and then **cancelled unrun**: its `share_bp` channel is identically zero for the same reason.

Current attempt — subjet-level features (k_t cut 20 GeV):

- ▶ `part_sj_lnm` — $\ln$ mass of my subjet
- ▶ `part_sj_lnptfrac` — subjet p_T fraction
- ▶ `part_sj_nconst` — constituents sharing it

Passes the set-model probe that killed the earlier variants. **Training result pending.**

Where the time goes — and what limits it

Nsight Systems — share of GPU time

kernel	% GPU	avg/call
_cluster_large_kernel	97.6	5.47 ms
_decode_kernel	1.5	82 μ s
torch glue	< 1	

Host↔device transfer negligible once running:
the memory row of the timeline is **empty**.

No hidden overhead — essentially all the time is real clustering work.

Nsight Compute — hardware counters

(H100)

metric

value

DRAM throughput

0.02 %

— Compute (SM) throughput

15.4 %

Registers / thread

168

Theoretical occupancy

18.75 %

Achieved occupancy

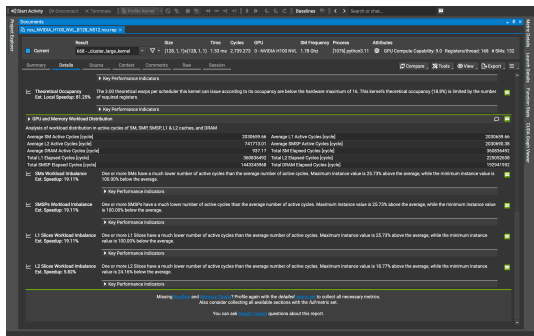
6.25 %

Waves per SM

0.32

Not memory- or compute-bound — the GPU is simply **not filled**.

The profiler names the bottleneck — and prices the fix



NVIDIA's own estimates:

finding

est. speedup

Theoretical occupancy (registers)

81%

Achieved occupancy

67%

SM workload imbalance

19%

"This kernel grid is too small to fill the available resources on this device, resulting in only 0.32 full waves across all SMs."

Grid size **128 blocks** on a **132-SM** device — fewer blocks than the GPU has processors.

~**1.7–1.8×** more available from launch configuration and register budget — **not** from the algorithm.

Summary

1. **flashjet clusters on the GPU and keeps the full merge history**, so substructure — exclusive subjets, soft drop, Lund coordinates — comes free as array reads rather than a second clustering pass.
2. **It gives the same jets as FastJet.** 150/150 grid points at 100 % n_{jets} agreement, across anti- k_t/k_t /C-A, $R = 0.4\text{--}1.2$, and 24–123 constituents per jet. k_t and C/A validated on real data for the first time.
3. **It reproduces ATLAS's own reconstructed jets** exactly, clustering ~ 600 calorimeter clusters per event.
4. **39–99 $\times$ faster** than vectorised FastJet on a V100S, and the advantage *grows* with jet complexity. Cost is independent of R and of the algorithm choice.
5. **Still a prototype**, with meaningful headroom: the profiler says the GPU is largely idle and attributes $\sim 1.7\text{--}1.8\times$ to register pressure alone.

What is next

Item	Why
1 C++ FastJet baseline	every number here uses the Python binding; the production tool is C++
2 Improve GPU utilisation	the profile says most of the GPU is idle; re-tuning launch geometry is free speed
3 Pile-up and grooming timing	untested axes that a realistic workload will hit
4 Use it inside experiment software	the real production test — needs a C++/CUDA rewrite

Bottom line: as an accelerator-resident substitute for CPU clustering inside a learning workflow, flashjet is **correct**, roughly **two orders of magnitude faster** than per-jet FastJet, and still leaving most of the GPU idle.

Thank you

Questions?

All benchmarks: ATLAS Open Data (CC0) — ATL-PHYS-PUB-2022-039 and the 2020 ATLAS Jet Reconstruction dataset.

Physics closures: analytic predictions on toy showers.

Code: `flashjet`, branch benchmarking.

Backup — reproducing this

```
# 1. ATLAS Open Data -> per-jet constituent lists (npz)
python dump_atlas_toptag.py {top|qcd} 30000

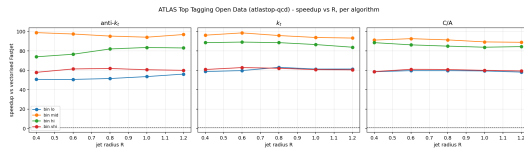
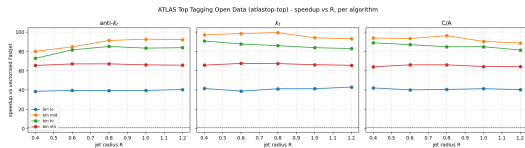
# 2. full sweep on a GPU: 3 algorithms x 5 radii x 5 bins
python sweep_flashjet.py atlastop-top 20000    # flashjet side
python sweep_fastjet.py  atlastop-top          # FastJet side + agreement

# 3. plots
python make_atlas_plots.py
```

Dataset	Source
Top Tagging Open Data	opendata.cern.ch/record/15013 (and 80030)
2020 Jet Reconstruction	opendata.cern.ch/record/15010

Both read directly over the open eospublic redirector — no grid certificate required. Constituents are calorimeter clusters, stored in MeV, massless.

Backup — speedup vs R , per algorithm

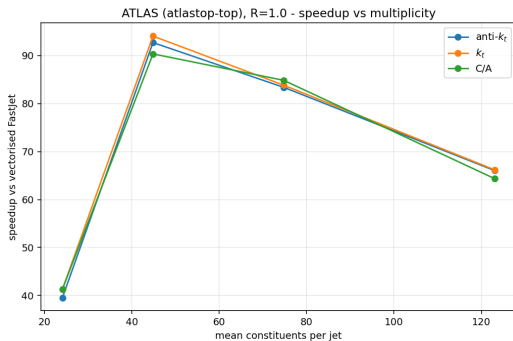


boosted top

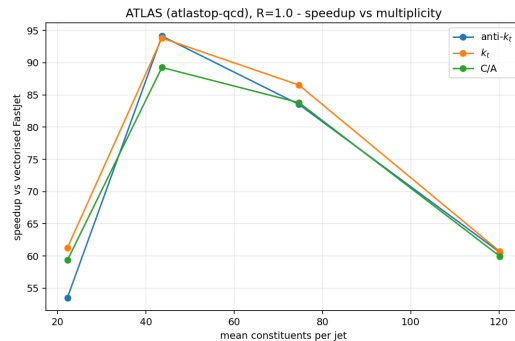
QCD (light q/g)

Flat in R for all three algorithms; the ordering is by multiplicity bin.

Backup — speedup vs multiplicity

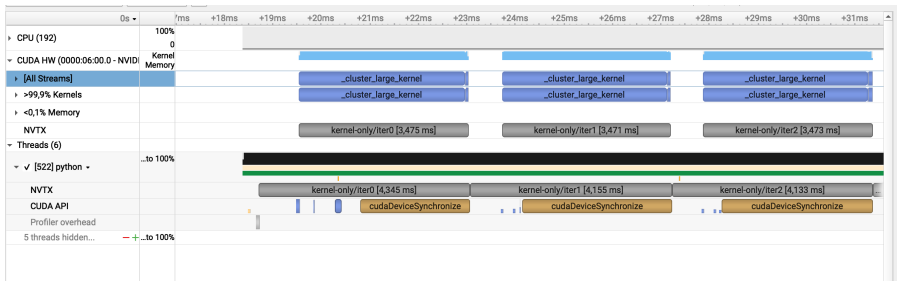


boosted top



QCD (light q/g)

Backup — the GPU timeline



Nsight Systems, three consecutive iterations. Each block is one clustering call; the tool labels the rows “>99.9% Kernels” and “<0.1% Memory”. The kernels run back to back with nothing in between, and the memory row is empty — no host/device shuffling while clustering runs. That is what makes the speedup real rather than an artefact of where the timer started.

Backup — occupancy detail

The screenshot shows the 'Occupancy' section of the NVIDIA Nsight Compute interface. It provides a detailed breakdown of occupancy metrics for a specific kernel. The text explains that occupancy is the ratio of active warps to the maximum possible warps, and higher occupancy generally leads to better performance. The table below lists various occupancy metrics:

Metric	Value	Unit
Theoretical Occupancy [%]	18.75	Block Limit Registers [Block]
Theoretical Active Wps per SM [warp]	12	Block Limit Shared Wps [Block]
Achieved Occupancy [%]	6.25	Block Limit Wps [Block]
Achieved Active Wps per SM [warp]	4.00	Block Limit SM [Block]
Cluster Occupancy [%]	0	Block Limit Shared Wps [Block]
Max Active Clusters [cluster]	0	Max Cluster Size [Block]
Overall GPU Occupancy [%]	0	

Below the table, there are two sections: 'Achieved Occupancy' and 'Theoretical Occupancy'. The 'Achieved Occupancy' section states that the difference between calculated theoretical (18.75%) and measured achieved occupancy (6.25%) can be the result of warp scheduling overheads or workload imbalances. The 'Theoretical Occupancy' section states that the 3.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 16. This kernel's theoretical occupancy (18.75%) is limited by the number of register registers.

The screenshot shows the 'Performance' section of the NVIDIA Nsight Compute interface. It provides a detailed breakdown of performance metrics for a specific kernel. The text explains that performance is the ratio of active warps to the maximum possible warps, and higher performance generally leads to better performance. The table below lists various performance metrics:

Metric	Value	Unit
GPU Speed of Light Throughput	15.42	Operation [ns]
Memory Throughput [%]	19.24	Repeat Count [Byte]
L1/L2 Cache Throughput [%]	23.23	SM Active Cycle [Byte]
L2 Cache Throughput [%]	0.82	SM Frequency [Hz]
Global Throughput [%]	0.02	Global Frequency [Hz]

Below the table, there are two sections: 'Launch Statistics' and 'Occupancy'. The 'Launch Statistics' section provides a summary of the configuration used to launch the kernel, including the number of registers per thread, the number of threads per block, and the number of blocks per grid. The 'Occupancy' section provides a detailed breakdown of occupancy metrics, including the theoretical occupancy, the achieved occupancy, and the overall GPU occupancy.

Register pressure (168/thread) caps theoretical occupancy at 18.75 %; achieved is 6.25 %. With 0.32 waves/SM the kernel does not fill the GPU even once. DRAM throughput 0.02 % — this is parallelism starvation, not a bandwidth problem.

Backup — why not PHYSLITE?

ATLAS's main research open-data format, **DAOD_PHYSLITE**, cannot be used for this benchmark: it stores calibrated *objects*, not jet-finding *inputs*.

looked for	found
CaloCalTopoClusters	0 branches
Flow-element / PFO containers	0 branches
general TruthParticles	absent
constituentLinks on jets	present but dangling
InDetTrackParticles	thinned to ~ 4 tracks/event

The links carry a *non-zero* container key with 22.5 constituents/jet recorded — the structure is described, but the four-vectors are not shipped. ATLAS confirm this is deliberate: “*we don't store jet constituents in PHYSLITE.*” Hence the datasets used here, which do ship constituents.